

Bài Tập Cuối Chương

- Trong các lựa chọn sau, yếu tố nào **quan trọng nhất** để xem xét khi quyết định phân chia giao diện người dùng thành các component nhỏ hơn trong React?
 - A. Giảm số lượng dòng code trong một file duy nhất để dễ quản lý.
 - B. Tăng cường khả năng tái sử dụng code, từ đó nâng cao hiệu quả bảo trì và tiềm năng cải thiện hiệu suất ứng dụng.
 - C. Đảm bảo ứng dụng tuân thủ theo đúng chuẩn mực thiết kế component của React.
 - D. Giúp ứng dụng chạy nhanh hơn bằng cách giảm tải cho trình duyệt.
- Tình huống nào sau đây là **lý do chính đáng nhất** để bạn tách một phần giao diện thành một component React độc lập?
 - A. Để dễ dàng áp dụng các style CSS khác nhau cho từng phần của giao diện.
 - B. Khi nhận thấy một đoạn mã JSX trong component trở nên quá dài và khó theo dõi.
 - C. Khi bạn dự đoán rằng phần giao diện đó có thể được sử dụng lại một cách nhất quán ở nhiều vị trí khác nhau trong ứng dụng, với các dữ liệu đầu vào (props) khác nhau.
 - D. Để đơn giản hóa logic của component cha bằng cách chuyển bớt JSX sang component con.
- Việc phân chia component **quá chi tiết**, tạo ra các component quá nhỏ và có phạm vi sử dụng hạn hẹp, có thể dẫn đến hậu quả tiêu cực nào?
 - A. Ứng dụng sẽ tải chậm hơn do phải tải nhiều file component nhỏ.
 - B. Cấu trúc dự án trở nên phức tạp và khó quản lý, gây trở ngại cho việc theo dõi luồng dữ liệu và bảo trì ứng dụng về lâu dài.
 - C. Giảm khả năng ứng dụng các kỹ thuật tối ưu hiệu suất re-render của React.
 - D. Không có hậu quả tiêu cực đáng kể, việc phân chia càng nhỏ càng tốt để dễ quản lý code.

Bài 4:

Bạn hãy tạo một giao diện danh sách sản phẩm **tối giản** cho một trang web bán hàng trực tuyến. Giao diện này sẽ trình bày một danh sách các sản phẩm. Thông tin hiển thị cho mỗi sản phẩm bao gồm:

- Ảnh sản phẩm (placeholder):** Sử dụng hình ảnh placeholder đơn giản (ví dụ: hình vuông màu xám hoặc một biểu tượng sản phẩm chung), không cần sử dụng dữ liệu ảnh thực tế.
- Tên sản phẩm:** Hiển thị rõ ràng tên của sản phẩm.
- Giá sản phẩm:** Hiển thị giá bán của sản phẩm, có thể kèm theo đơn vị tiền tệ (ví dụ: VNĐ, USD).
- Nút "Xem chi tiết":** Tạo một nút bấm với nội dung "Xem chi tiết" cho mỗi sản phẩm. Không cần thực hiện chức năng "nhấp để xem chi tiết", chỉ cần nút bấm hiển thị đúng giao diện là đủ.

Yêu cầu bài tập:

- Phân chia Component Hợp Lý (Nguyên tắc Tái Sử Dụng):** Hãy suy nghĩ và phân chia giao diện thành các component React một cách hợp lý nhất, dựa trên nguyên tắc tái sử dụng. Cân nhắc xem phần nào của giao diện có thể được tái sử dụng cho các sản phẩm khác nhau, và phần nào nên được giữ lại trong một component lớn hơn. Tránh việc chia nhỏ component quá mức không cần thiết.
- Component `ProductCard` (Bắt Buộc, Tính Tái Sử Dụng):** Bắt buộc tạo một component tái sử dụng với tên gọi `ProductCard`. Component này phải chịu trách nhiệm hiển thị layout thông tin cơ bản của một sản phẩm đơn lẻ (ảnh, tên, giá, nút "Xem chi tiết"). `ProductCard` cần được thiết kế để có thể tái sử dụng cho nhiều sản phẩm khác nhau và nhận dữ liệu thông tin sản phẩm thông qua props.
- Component `ProductList` (Bắt Buộc, Quản Lý Danh Sách Sản Phẩm):** Bắt buộc tạo component `ProductList`. Component này sẽ đóng vai trò là container để quản lý và hiển thị danh sách các component `ProductCard`. `ProductList` sẽ nhận một mảng dữ liệu sản phẩm (ví dụ: một mảng các object JavaScript chứa thông tin sản phẩm mẫu, được định nghĩa tĩnh trong code) và sử dụng phương thức `.map()` của JavaScript để render một

danh sách các component `ProductCard` , truyền dữ liệu sản phẩm tương ứng cho từng `ProductCard` thông qua props.

- **Dữ liệu Sản Phẩm Mẫu (Tĩnh, Tối Thiểu 3 Sản Phẩm):** Sử dụng dữ liệu sản phẩm mẫu dạng tĩnh (được hardcode trực tiếp trong file component, không cần dữ liệu từ API). Hãy tạo một mảng JavaScript chứa thông tin của ít nhất 3 sản phẩm mẫu khác nhau. Mỗi object sản phẩm trong mảng cần có tối thiểu các thuộc tính sau: `id` , `name` , `price` , và `image` (URL của ảnh placeholder).
- **Cấu Trúc Thư Mục Dự Án Chuẩn:** Tổ chức các file component theo cấu trúc thư mục dự án React phổ biến. Cụ thể, đặt các component `ProductCard` và `ProductList` vào thư mục con `src/components` . Component `App` có thể được giữ ở thư mục `src/` gốc.
- **Style Giao Diện Cơ Bản (Khuyến Khích, Không Bắt Buộc):** Khuyến khích bạn tự do thêm một chút style CSS cơ bản (ví dụ: sử dụng inline style trực tiếp trong JSX hoặc CSS modules) để giao diện danh sách sản phẩm và từng sản phẩm con hiển thị rõ ràng, dễ nhìn và có bố cục phân biệt các phần. Không yêu cầu thiết kế giao diện đẹp mắt hoặc quá phức tạp. Mục tiêu chính là để giao diện đủ rõ ràng cho mục đích kiểm tra khả năng phân chia component của bạn.

Gợi ý về cách triển khai:

- **Bắt đầu với dữ liệu mẫu:** Khởi tạo một mảng `productsData` chứa các object sản phẩm mẫu.
- **Component `ProductCard` :** Tạo file `src/components/ProductCard.jsx` (và `ProductCard.css` nếu dùng CSS modules) để xây dựng cấu trúc hiển thị cho một sản phẩm.
- **Component `ProductList` :** Tạo file `src/components/ProductList.jsx` (và `ProductList.css` nếu dùng CSS modules) để hiển thị danh sách các `ProductCard` .
- **Component `App` :** Trong `App.jsx` , import và sử dụng component `ProductList` , truyền dữ liệu `productsData` vào thông qua prop `products` .

Bài 5:

Bạn đang phát triển giao diện trang **Tổng Quan Sản Phẩm** cho ứng dụng quản lý kho. Trang này cần hiển thị các thông tin sản phẩm một cách **thích ứng** với các tình huống khác nhau. Giao diện trang bao gồm các phần chính:

1. **Tiêu đề Trang:** Luôn hiển thị dòng tiêu đề **"Tổng Quan Sản Phẩm"**.
2. **Khu vực Thống kê Sản Phẩm (Dashboard Summary - Chỉ Admin):**
 - Hiển thị các số liệu thống kê quan trọng về sản phẩm, bao gồm:
 - Tổng số sản phẩm hiện có trong kho.
 - Số lượng sản phẩm đang còn hàng.
 - Số lượng sản phẩm hết hàng.
 - Điều kiện hiển thị: Khu vực thống kê này chỉ hiển thị cho người dùng có vai trò là "admin". Người dùng có vai trò "user" sẽ không nhìn thấy khu vực thống kê này trên trang tổng quan.
3. **Bảng Danh Sách Sản Phẩm (Luôn Hiển Thị - Thay Đổi theo Vai trò):**
 - Hiển thị một bảng chứa danh sách các sản phẩm.
 - Bảng danh sách sản phẩm này phải luôn hiển thị, bất kể vai trò người dùng hay trạng thái tải dữ liệu.
 - Cột "Hành động" (Actions - Chỉ Admin):
 - Điều kiện hiển thị: Cột "Hành động" (chứa các nút chức năng như "Sửa" và "Xóa" cho từng sản phẩm) chỉ được hiển thị cho người dùng có vai trò "admin".
 - Người dùng "user" sẽ không nhìn thấy cột "Hành động" trong bảng danh sách sản phẩm, bảng của họ chỉ hiển thị thông tin sản phẩm cơ bản.
4. **Thông báo Trạng thái (Loading và Lỗi):**
 - Thông báo "Đang tải dữ liệu sản phẩm..." (Loading State): Hiển thị dòng thông báo này trong khi ứng dụng đang trong quá trình lấy dữ liệu sản phẩm từ nguồn dữ liệu (ví dụ: API).
 - Thông báo "Lỗi tải dữ liệu. Vui lòng kiểm tra lại kết nối hoặc thử lại sau." (Error State): Hiển thị thông báo lỗi này nếu có lỗi xảy ra trong quá trình tải dữ liệu sản phẩm.
 - Điều kiện hiển thị thông báo trạng thái:
 - Thông báo "Đang tải..." hiển thị khi giá trị state `isLoading` là `true` .

- Thông báo "Lỗi tải dữ liệu..." hiển thị khi state `error` không phải `null` (tức là có lỗi), và bảng danh sách sản phẩm không được hiển thị khi có lỗi (chỉ hiển thị thông báo lỗi).
- Khi quá trình tải dữ liệu thành công (không lỗi và `isLoading` là `false`), bảng danh sách sản phẩm sẽ được hiển thị, và các thông báo trạng thái (loading, lỗi) sẽ bị ẩn.

Yêu cầu Bài Tập:

Viết code JSX trong React để render trang Tổng Quan Sản Phẩm này, chú trọng sử dụng các phương pháp render có điều kiện đã học để đáp ứng các yêu cầu hiển thị khác nhau dựa trên vai trò người dùng, trạng thái tải, và lỗi.

Bạn cần viết code cho các phần sau:

- 1. Khu vực Thống kê Sản Phẩm (Dashboard Summary): Viết code JSX để hiển thị có điều kiện khu vực thống kê (bao gồm tiêu đề và các số liệu thống kê) dựa trên giá trị của state `userRole` (chỉ hiển thị khi `userRole` là `"admin"`).
- 2. Cột "Hành động" (Actions) trong Bảng Sản Phẩm: Viết code JSX để render có điều kiện cột tiêu đề "Hành động" và các ô dữ liệu "Hành động" (`<td>`) chứa nút "Sửa" và "Xóa" trong bảng danh sách sản phẩm, dựa trên giá trị của state `userRole` (chỉ hiển thị cột "Hành động" khi `userRole` là `"admin"`).
Lưu ý: Bạn cần giả định rằng bạn đã có sẵn code JSX cơ bản để tạo bảng danh sách sản phẩm (ví dụ: sử dụng thẻ `<table>` , `<thead>` , `<tbody>` , `<tr>` , `<th>` , `<td>`) và bạn chỉ cần thêm logic render có điều kiện cho cột "Hành động" vào code bảng đó.
- 3. Thông báo "Đang tải dữ liệu sản phẩm..." (Loading): Viết code JSX để hiển thị có điều kiện thông báo "Đang tải dữ liệu sản phẩm..." dựa trên giá trị của state `isLoading` .
- 4. Thông báo "Lỗi tải dữ liệu. Vui lòng kiểm tra lại kết nối hoặc thử lại sau." (Error): Viết code JSX để hiển thị có điều kiện thông báo lỗi dựa trên giá trị của state `error` . Đảm bảo rằng bảng danh sách sản phẩm không hiển thị khi có lỗi, mà thay vào đó chỉ hiển thị thông báo lỗi.

Giả định State (đã có sẵn trong component Trang Tổng Quan Sản Phẩm):

- `isLoading` : Một biến state kiểu boolean. Giá trị `true` khi dữ liệu sản phẩm đang được tải, và `false` khi quá trình tải hoàn tất hoặc chưa bắt đầu.
- `error` : Một biến state đại diện cho lỗi. Giá trị có thể là `null` (nếu không có lỗi) hoặc một object lỗi (ví dụ: `Error` object) nếu có lỗi xảy ra trong quá trình tải dữ liệu.
- `products` : Một biến state kiểu mảng. Là một mảng chứa dữ liệu sản phẩm (mỗi phần tử trong mảng là một object mô tả thông tin của một sản phẩm). Bạn có thể khởi tạo giá trị ban đầu của `products` là một mảng rỗng (`[]`).
- `userRole` : Một biến state kiểu string. Giá trị có thể là `"admin"` hoặc `"user"` . Biến này xác định vai trò của người dùng hiện tại đang xem trang tổng quan sản phẩm. Bạn có thể giả định giá trị ban đầu của `userRole` là `"admin"` hoặc `"user"` để thuận tiện cho việc kiểm thử các điều kiện render khác nhau.

Dữ liệu Sản Phẩm Mẫu (tham khảo - dùng để hiển thị bảng):

Bạn có thể sử dụng dữ liệu sản phẩm mẫu **dưới đây** để hiển thị trong bảng danh sách sản phẩm (hoặc tự tạo dữ liệu mẫu khác):

```
const productsData = [  
  { id: 1, name: 'Laptop XYZ', price: 1200, stock: 15 },  
  { id: 2, name: 'Điện thoại ABC', price: 800, stock: 0 },  
  { id: 3, name: 'Máy tính bảng DEF', price: 500, stock: 20 },  
];
```

6. Điều gì xảy ra khi một component React re-render?
- A. Component bị xóa khỏi DOM và tạo lại hoàn toàn.
 - B. Chỉ những phần tử DOM có sự thay đổi mới được cập nhật.
 - C. Function component được gọi lại, JSX được tạo lại, và React so sánh Virtual DOM để cập nhật DOM thật.
 - D. Chỉ state của component được cập nhật, giao diện không thay đổi.
-
7. Theo cơ chế re-render mặc định của React, điều gì sẽ xảy ra với component con khi component cha của nó re-render?

- A. Component con sẽ không re-render trừ khi props của nó thay đổi.
- B. Component con cũng sẽ re-render, ngay cả khi props của nó không thay đổi.
- C. Chỉ khi component con sử dụng state của component cha thì nó mới re-render.
- D. Việc component con re-render hay không là ngẫu nhiên và không thể dự đoán.

8. Nguyên tắc "Colocate State" (Đặt State gần nơi sử dụng nhất) trong React nhằm mục đích chính là gì?

- A. Giảm thiểu re-render không cần thiết của các component con, từ đó cải thiện hiệu suất ứng dụng.
- B. Làm cho code component dễ đọc và dễ hiểu hơn.
- C. Đơn giản hóa việc truyền dữ liệu giữa các component cha con.
- D. Tăng khả năng tái sử dụng state trong ứng dụng.

9. Khi nào thì việc đặt state ở component cha chung gần nhất (nguyên tắc "Colocate State") trở nên không phù hợp?

- A. Khi component cần sử dụng state có quá nhiều props.
- B. Khi state đó cần được sử dụng và cập nhật bởi nhiều component khác nhau, nằm ở các nhánh khác nhau của cây component.
- C. Khi component cha chung gần nhất có logic quá phức tạp.
- D. Khi muốn tối ưu hiệu suất re-render bằng mọi giá.

10. Giải pháp nào thường được sử dụng khi state cần được chia sẻ và sử dụng bởi nhiều component không trực tiếp liên quan đến nhau trong một ứng dụng React lớn?

- A. Prop drilling (truyền props qua nhiều tầng component).
- B. Component Composition (kết hợp component).
- C. Context API hoặc các thư viện quản lý state toàn cục (ví dụ: Redux, Zustand).
- D. Sử dụng ref để truy cập và thay đổi state trực tiếp giữa các component.

11. Trong React functional component, Hook `useEffect` được sử dụng để làm gì?

- A. Để khai báo và quản lý state của component.
- B. Để tối ưu hiệu suất re-render của component.
- C. Để thực hiện và quản lý "side effects" (tác dụng phụ) trong vòng đời của component.
- D. Để tạo ra các component có khả năng tái sử dụng cao hơn.

12. Điều gì KHÔNG phải là một ví dụ về "side effect" trong React component?

- A. Gọi API để lấy dữ liệu từ server.
- B. Thiết lập một bộ đếm thời gian (timer) bằng `setInterval`.
- C. Thay đổi trực tiếp DOM bằng JavaScript.
- D. Tính toán và trả về JSX để render giao diện người dùng.

13. Khi bạn sử dụng `useEffect` với dependency array rỗng `[]`, callback function bên trong `useEffect` sẽ được thực thi khi nào?

- A. Sau mỗi lần component re-render.
- B. Chỉ một lần duy nhất sau lần render đầu tiên của component (tương tự `componentDidMount`).
- C. Khi component unmount (bị hủy).
- D. Khi có bất kỳ prop hoặc state nào của component thay đổi.

14. Cleanup function (function được return từ callback của `useEffect`) có vai trò chính là gì?

- A. Để tối ưu hiệu suất của `useEffect` Hook.
- B. Để "dọn dẹp" các side effects đã thiết lập trước đó, ví dụ hủy timers hoặc subscriptions, và ngăn ngừa memory leaks.

- C. Để gọi API fetch dữ liệu một cách hiệu quả hơn.
 - D. Để thông báo cho React biết khi nào component cần re-render.
-

15. Điều gì sẽ xảy ra nếu bạn sử dụng `useEffect` mà **không truyền vào dependency array** (để trống đối số thứ hai)?

- A. `useEffect` callback function sẽ không bao giờ được thực thi.
- B. `useEffect` callback function chỉ được thực thi một lần duy nhất khi component mount.
- C. `useEffect` callback function sẽ được thực thi sau mỗi lần render của component.
- D. `useEffect` sẽ hoạt động giống như `React.memo` để tối ưu hiệu suất.